

RoTino: Modelling and Control of a Wheeled Bipedal Robot

A cascaded PID built around the Zero Moment Point
versus a decoupled MPC + TV-LQR architecture

Alessandro Prisco

Aldo Rosicarelli

Field and Service Robotics — Final technical project, a.y. 2025/2026
University of Naples Federico II — October 2026

Abstract

RoTino is a 4.3 kg wheeled bipedal robot: two articulated legs, each ending in a driven wheel, carry a torso that must be kept in balance like an inverted pendulum. The robot is simulated in Gazebo with ROS 2 and torque control on its six joints. This report describes how it was modelled and how two control laws of very different nature were designed, implemented and compared on it. The first is a cascaded PID in which the Zero Moment Point is the variable that the wheels control: a capture-point loop decides where the contact should be with respect to the centre of mass, and a preview of the planned path makes the robot lean before it accelerates. The second re-implements the decoupled architecture of Cui *et al.*: a model predictive controller plans the motion of the upper body, a gain-scheduled LQR balances the wheels and virtual model control drives the legs. The two laws are run on the same robot, world and scenarios—pushes, constant forces, reference tracking, curved paths, uneven ground and a jump—and the differences in the results are traced back to the design choices that cause them.

1 Introduction

Wheels are fast and efficient on flat ground; legs adapt to obstacles and let a robot change the height and the attitude of its body. A wheeled bipedal robot puts a driven wheel at the end of each leg and tries to have both [1, 2]. The price is balance. The robot touches the ground only along the line that joins the two wheels, so it is an inverted pendulum that has to be stabilised at every instant, and it has fewer actuators than degrees of freedom.

This is what makes the platform attractive for this course. A single small robot lies at the crossing of the chapter on wheeled robots and of the chapter on legged robots: its wheels roll without slipping, so its motion on the plane obeys nonholonomic constraints; its torso is a floating base that can only be moved through the forces exchanged with the ground; its support polygon degenerates into a segment, so the Zero Moment Point (ZMP) takes a special role; and it is underactuated, so translation can only be obtained by tilting, much as a quadrotor does. Although two wheels give the planar motion of a differential drive, the project is not about its kinematic control: the kinematic layer reduces to a simple heading guidance, and the whole difficulty lies in the underactuated dynamics of balance.

The robot, named RoTino, is simulated in Gazebo Fortress with ROS 2 Humble. Two control laws were developed for it. The first is a cascade of PID loops that was redesigned around the ZMP. The second follows the paper that guided the project, *Modeling and Control of a Wheeled Biped Robot* by Cui *et al.* [1]: the robot is decoupled into a wheeled inverted pendulum of variable length, balanced by a time-varying LQR (TV-LQR), and a lumped-mass upper body, moved by a model predictive controller (MPC) through virtual model control of the legs. Having both on the same plant allows a question to be asked that neither answers alone: what does the model-based, optimisation-based architecture buy with respect to a well-designed cascade, and what does it cost?

Table 1: Main parameters of RoTino, computed from the URDF by the model library (rounded).

total mass	M	4.3 kg	thigh and shank length	$l_1 = l_2$	0.13 m
mass above the wheels	m_b	3.5 kg	wheel radius	r	0.06 m
mass of one wheel	m_w	0.38 kg	distance between the wheels	d	0.29 m
pendulum length (nominal)	l	0.16 m	hip height (nominal)		0.24 m
CoM height (nominal)	h	0.19 m	wheel-ground friction	μ	2.2
wheel torque used		± 10 Nm	leg torque used		± 50 Nm

Starting point and novelty. The mechanical model of the robot was available as a MuJoCo description, together with a balance-and-jump controller that commanded the legs through position servos. The model was converted to URDF with a torque interface on every joint, and the controller was ported to ROS 2. Ported as it was, it did not survive the change: with the leg loop closed in a ROS node at 500 Hz rather than inside the simulator, the legs entered a limit cycle. Everything else described in this report was developed in the project:

- a ROS 2 workspace in which the simulation, the robot, the scenarios and the telemetry are shared and only the controller node changes, with a model library that computes every parameter of the models from the URDF (Section 2);
- the modelling of the robot: configuration space and constraints, a planar Lagrangian model, and the two reduced models used for control with the parameters of RoTino (Section 3);
- a cascaded PID controller designed around the ZMP and the capture point, with a preview of the planned path and a gain design supported by a robustness analysis (Section 4);
- an implementation of the architecture of Cui *et al.* in which every numerical block—Riccati solver, condensed quadratic program, Kalman filter—is written from scratch, with several adaptations that the small size of the robot and the torque-controlled simulation required (Section 5);
- a benchmark of twelve scenarios run with both laws, in which the ZMP is reconstructed from the multibody dynamics because the simulator does not provide contact forces, and a live dashboard (Section 6).

The two authors worked jointly on every part of the project—modelling, both controllers, the benchmark and the analysis of the results—so no separate attribution of the work is made.

2 The robot and the software workspace

2.1 RoTino

RoTino (Fig. 1a) has a box-shaped torso and two identical legs. Each leg is a planar chain of a thigh and a shank of equal length, with a hip and a knee joint whose axes are parallel to the wheel axle, and ends in a driven wheel. All six joints are torque controlled. In the nominal posture thigh and shank are at $\pm 45^\circ$, so the wheel axle sits exactly below the hips. Table 1 lists the parameters that matter for control. They are not typed into the controllers: a model library reads them from the URDF, so that the description of the robot is the only place where they are stored.

Two features of this small robot shape the control problem. First, about four fifths of the mass sit above the wheels and close to them: the equivalent pendulum is only 16 cm long, which makes the unstable dynamics fast. Second, the centre of mass (CoM) of the upper body is not above the hips but about one centimetre ahead of them. With the wheels under the hips the equivalent pendulum already leans by almost 5° : a controller cannot assume that “torso upright” means “balanced”, and has to find the equilibrium by itself.

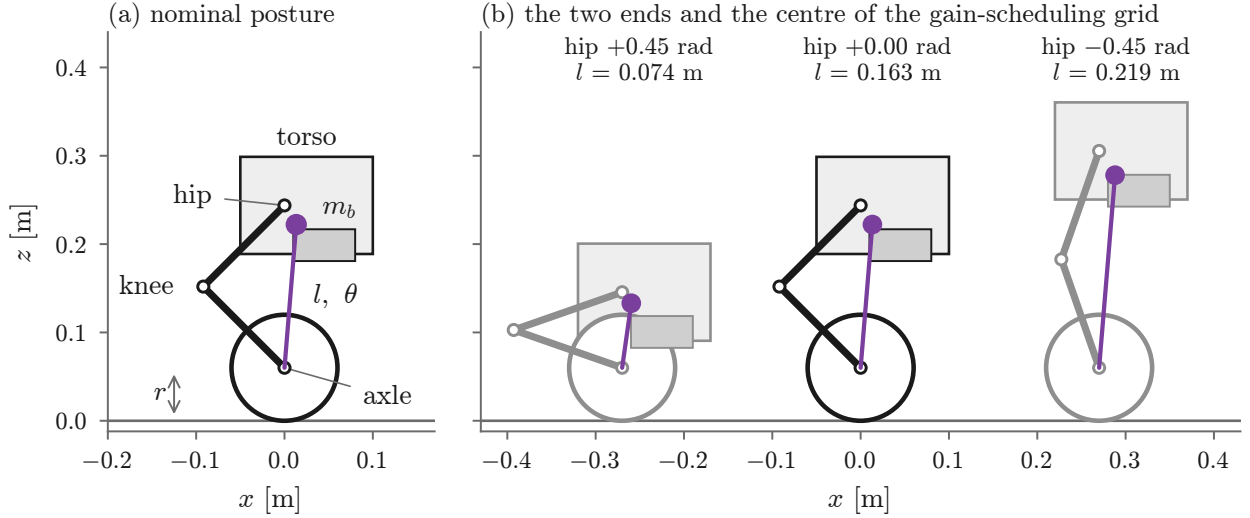


Figure 1: Sagittal view of RoTino, drawn to scale from the URDF. The purple segment is the equivalent pendulum used for control: it joins the wheel axle to the centre of mass m_b of everything above the wheels, with length l and lean angle θ from the vertical. Folding or stretching the legs changes l by a factor of three.

2.2 Simulation set-up

The robot runs in Gazebo Fortress with a physics step of 0.5 ms. The six joints are exposed through `ros2_control` as effort interfaces and are commanded at 500 Hz, the rate of every loop of this report unless stated otherwise. The sensors are the joint encoders, an IMU on the torso, a contact sensor on each wheel and, for analysis, a plugin that publishes the true pose of the torso. In this version of the simulator the contact sensors report that a wheel is touching the ground, but not the force it exchanges: this limitation is the reason why the ZMP is reconstructed rather than measured (Section 6.1).

Every run starts in the same way. The robot is spawned with the torso welded to a fixed anchor; the controller brings the legs to the standing posture and then publishes a release message that detaches the anchor. From that moment the robot is on its own, and time in all the plots is counted from the release. External disturbances are applied to the torso through the simulator: an impulsive push and a constant force.

The world (Fig. 2) is a flat arena with three platforms of uneven ground placed away from the origin: three speed bumps, a 5 % ramp leading to a raised plateau, and a field of tiles of random height that the two wheels cross on different columns, so that the left and the right wheel meet different steps.

2.3 Organisation of the workspace

The workspace `rotino_ws` contains five ROS2 Python packages (Table 2, Fig. 3). Their organisation follows one rule: *when two controllers are compared, nothing but the controller may differ*. Four choices enforce it.

One launch file. The simulation, the robot and the scenario are started by a single launch file in `rotino_description`; each control package includes it and passes only the name of its own executable. The scenarios—a trapezoidal speed profile, a back-and-forth motion, an S-shaped path, a change of height, a push, a jump—are launch arguments that both controllers understand in the same way.

One model. Kinematics, masses and the reduced models are computed in one library from the URDF. Both controllers, the dashboard and the benchmark import the same functions, so a difference in the results cannot come from a difference in the model.

One telemetry. Both controllers publish the same debug vectors (pitch, position, speed, refer-

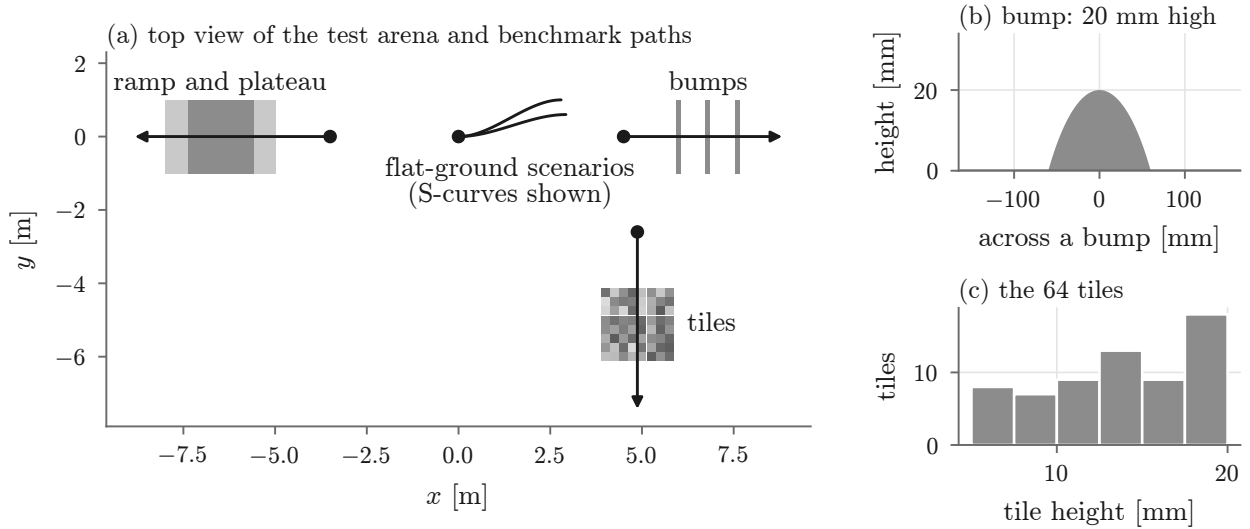


Figure 2: The test arena. Dots mark where the robot is released and arrows the path it is asked to follow in each uneven-ground scenario; the flat-ground scenarios start from the origin. Tiles are shaded by height.

Table 2: The packages of the workspace.

Package	Content
<code>rotino_description</code>	URDF of the robot, Gazebo world, controller configuration and the common launch file; the shared model library: forward kinematics and CoM from the URDF, the reduced models of Section 3.3, the multibody ZMP and the reference trajectories.
<code>rotino_pid</code>	The cascaded PID with the ZMP (Section 4): the balance law as a pure NumPy module with its offline tests, and the ROS node with heading control, leg control and jump supervisor.
<code>rotino_mpc</code>	The MPC + TV-LQR + VMC controller (Section 5): the numerical blocks (Riccati solver, gain schedule, condensed QP, Kalman filter) and the ROS node.
<code>rotino_benchmark</code>	The catalogue of scenarios, the scripts that run them headless with both laws, the 500 Hz logger, the step-force node, the metrics, the ZMP analysis and the comparison plots.
<code>rotino_dashboard</code>	A real-time dashboard (states, torques, phases, top view with the ZMP) that also sends live commands: speed, turn rate, height, jump and push.

ences, commanded offsets and torques), so one logger and one set of metrics serve both.

No black boxes. The numerical parts are written in plain NumPy and do not depend on ROS: the Riccati equation, the quadratic programs, the Kalman filter and the filters used for the ZMP are implemented in the workspace rather than taken from a control library. This makes them readable and testable offline: about forty unit tests check the balance law on the linear model, the ZMP reconstruction on cases with a known answer and the benchmark metrics.

In practice, once the workspace is built, one command starts the simulation with a controller and a scenario, and one command runs the whole comparison and writes its tables and plots:

```
ros2 launch rotino_pid rotino_pid.launch.py velocity_enable:=true velocity_max:=1.0
ros2 launch rotino_mpc rotino_mpc.launch.py push_enable:=true dashboard:=true
ros2 run rotino_benchmark suite
```

The dashboard (Fig. 4) shows the same quantities live and can drive the robot by hand, which was the quickest way to find the behaviours worth turning into benchmark scenarios.

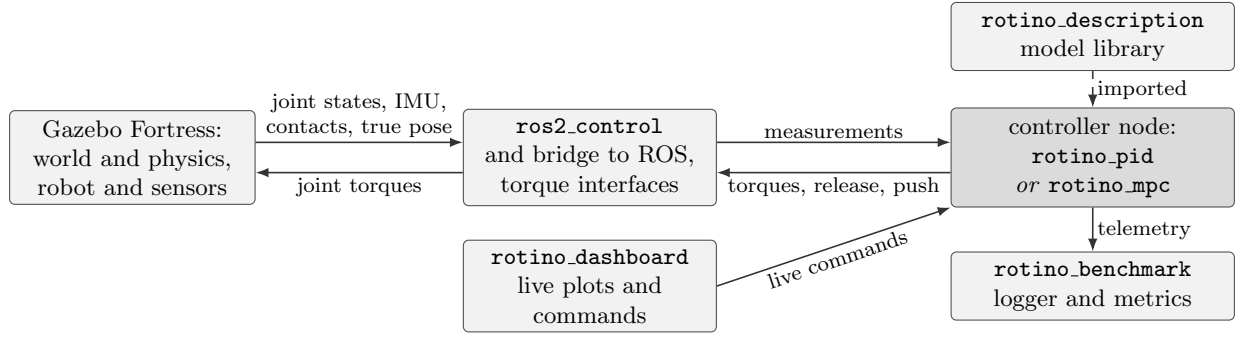


Figure 3: Structure of the workspace at run time. Only the shaded node changes between the two control laws; the model library is imported by the controllers, by the benchmark and by the dashboard.

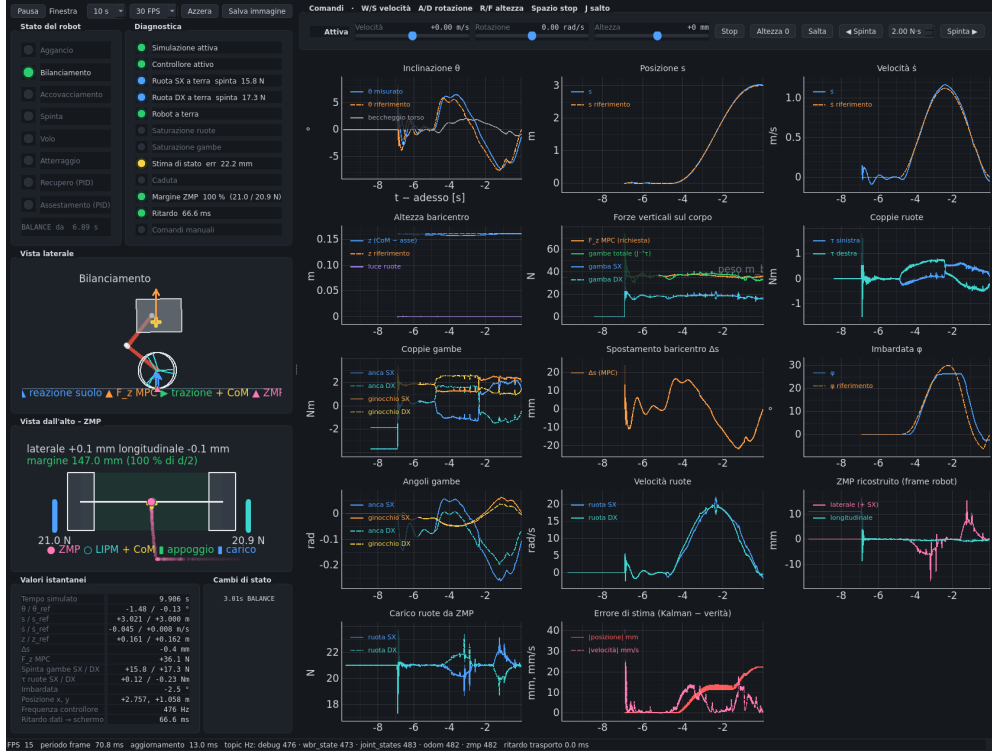


Figure 4: The dashboard during a fast S-curve with the MPC controller (the labels of the interface are in Italian). Left: phase and diagnostic indicators, side view of the robot with the forces, top view with the ZMP between the two wheels. Right: rolling plots of states, references, torques, wheel loads and estimation error.

3 Modelling

3.1 Configuration space, constraints and underactuation

Before any contact is imposed, the robot is a floating base with six joints. The torso is a rigid body in space with six degrees of freedom (DoF), and each of the two hips, two knees and two wheels adds one: twelve in total. The shape of the configuration space follows from the nature of each coordinate. The pose of the torso lives in $SE(3)$; hips and knees have mechanical limits, so each spans a closed interval rather than a circle; the wheels turn indefinitely, so together they span a torus. The configuration space is therefore $SE(3) \times I^4 \times T^2$, where I is an interval. For the orientation of the torso the code uses quaternions, an implicit representation that avoids the singularities of a three-angle parameterisation.

Contact with the ground adds constraints of two kinds, and the distinction matters because only one of them reduces the configuration space. With both wheels on flat ground, the height and the roll of the torso are no longer free: they follow from the pitch and from the leg joints. These

Table 3: Degrees of freedom n and actuators m of the models of RoTino.

Model	n	m	Free coordinates
robot on the ground, rolling	7	6	forward position, yaw, pitch, four leg joints
sagittal planar model	4	3	shank, wheel, knee, torso
wheeled pendulum (sagittal)	2	1	forward position, pitch

constraints are *holonomic*. Rolling without slipping is different. The two wheels impose that the axle midpoint cannot move sideways and that its forward speed $\dot{s}$ and the yaw rate $\dot{\varphi}$ are tied to the wheel speeds ω_l, ω_r :

$$\dot{x} \sin \varphi - \dot{y} \cos \varphi = 0, \quad \dot{s} = \frac{r}{2}(\omega_l + \omega_r), \quad \dot{\varphi} = \frac{r}{d}(\omega_r - \omega_l). \quad (1)$$

The first relation is not integrable: it restricts the velocities, not the positions that can be reached, and is a *nonholonomic* constraint in Pfaffian form. On the plane RoTino therefore moves like a differential drive. Two consequences are used later: the position of the robot can be reconstructed by integrating (1) (the odometry of Section 5.2), and a lateral error can be corrected only while moving (the guidance of Section 4.5).

Underactuation, as stressed in the course, is a property of the model used to describe the robot rather than of the robot itself, so it is worth counting it at the three levels at which RoTino is modelled (Table 3). At every level exactly one degree of freedom has no actuator, and it is always the same one: the pitch of the body about the wheel axle. No motor acts between the body and the ground. A wheel motor pushes the wheel forward and, by reaction, the body backward; this inertial coupling is the only handle on the pitch.

3.2 Planar Lagrangian model

When both legs move together the robot reduces to a planar chain: wheel, shank, thigh and torso. With the absolute angle of the shank q_1 , the rotation of the wheel q_w , the knee angle q_2 and the pitch of the torso q_3 as coordinates, the Euler–Lagrange equations take the usual form

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q) = B \tau, \quad \tau = (\tau_w, \tau_{knee}, \tau_{hip}). \quad (2)$$

This model was derived symbolically for RoTino; here it is used for what it shows rather than for its expressions. The matrix B has three columns for four coordinates, and it was built from the virtual work of the motors: each motor acts between two bodies, so its column is the gradient of the relative angle between them. A simple test follows from this. A rigid rotation of the whole robot about the axle changes no relative angle, hence no motor can do work on it: the coordinate associated with that motion—the pitch of the body—is passive. In the language of partial feedback linearisation [3], the pitch can only be controlled through a non-collocated scheme, by using the actuated coordinates to produce the inertial forces that move the passive one. Both controllers of this report do exactly this, each in its own way. The planar model also served as an independent check of the model library: the two give the same position of the CoM for the same posture.

3.3 The reduced models used for control

Neither controller uses (2) directly. Following Cui *et al.* [1], the robot is split into two simple models, each of which captures one part of the problem.

Equivalent pendulum. Everything above the wheels—torso, thighs and shanks—is replaced by its centre of mass. Its position with respect to the axle, computed by the library from the joint angles through the forward kinematics, defines a pendulum of length l and lean angle θ (Fig. 1). When the legs fold or stretch, l changes from 7 to 22 cm: the pendulum has a *variable length*.

Wheeled inverted pendulum. The wheels and this pendulum form the variable-length wheeled inverted pendulum (VL-WIP). Its state collects the forward position s of the axle, the lean θ , the yaw φ and their rates; the inputs are the two wheel torques. Linearised about the upright equilibrium it reads

$$\ddot{s} = a_1(l) \theta + b_1(l) (\tau_l + \tau_r), \quad \ddot{\theta} = a_2(l) \theta + b_2(l) (\tau_l + \tau_r), \quad \ddot{\varphi} = b_3 (\tau_r - \tau_l), \quad (3)$$

with the coefficients given in closed form in [1] as functions of the masses, of the wheel inertia and of l . Equation (3) already tells most of the story. The sum of the torques drives balance and forward motion together, their difference drives the yaw alone: it is natural to work with the common torque $\tau_c = (\tau_l + \tau_r)/2$ and the differential torque $\tau_d = (\tau_r - \tau_l)/2$. Moreover $a_2 > 0$ and $b_2 < 0$: the lean is unstable, and a forward torque tilts the body backward.

Two numbers summarise the sagittal dynamics of RoTino (Fig. 5a). The unstable pole is at about 10 rad/s: left alone, a lean doubles in less than a tenth of a second. The transfer function from the common torque to the position has a zero in the right half-plane at about 6 rad/s: the system is *non-minimum phase*. Physically, to move forward the wheels must first roll backward, so that the body falls forward, and only then accelerate. Any controller has to respect this order of events; the two controllers of this report differ precisely in how they plan it.

Lumped-mass upper body. The second model describes how the CoM moves when it is displaced from the wheels. Let δ be the horizontal distance of the CoM ahead of the contact point, h its height and F_z the vertical force of the legs. If the resultant of gravity and inertia is to pass through the contact, as it must since the wheel cannot transmit a moment,

$$\ddot{c} = \frac{g + \ddot{z}}{h} \delta, \quad \ddot{z} = \frac{F_z}{m_b} - g, \quad (4)$$

where $c = s + \delta$ is the horizontal position of the CoM. The first relation is the centrepiece of both controllers. With $\ddot{z} = 0$ it is the linear inverted pendulum, $\ddot{c} = \omega^2(c - p)$ with $\omega = \sqrt{g/h}$ and p the position of the contact: the CoM accelerates away from the contact, in proportion to the offset. For RoTino $\omega \approx 7$ rad/s. Whoever controls δ controls the acceleration of the robot; the MPC calls it Δs and optimises it over a horizon, the PID calls it the ZMP offset and regulates it with a cascade.

Differences from the paper. The structure of the models is that of [1]; the numbers are those of RoTino and come from the URDF. One departure is worth noting. The paper assigns to the upper body the inertia of a uniform rod, $m_b l^2/3$. For RoTino, whose torso is compact and sits right above the hips, this estimate is almost twice the real value in the nominal posture and less than half of it when the robot squats. The library therefore computes the inertia of the upper body about its own CoM for each posture, and the controllers use that value. The sign of the yaw equation was also adapted to the axes convention of ROS.

4 First controller: a cascaded PID built around the ZMP

4.1 The ZMP as the control input

For a legged robot the ZMP is usually a constraint: the point where the ground reaction produces no horizontal moment must stay inside the support polygon, and motions are planned so that it does [4]. On two wheels the picture changes. The support polygon is the segment between the two contacts. Sideways the ZMP can move along it and must remain inside it, as usual. In the direction of travel, however, the segment has no width: the ZMP necessarily lies on the contact line, and the wheels decide where that line is. The ZMP is no longer a quantity to be kept within

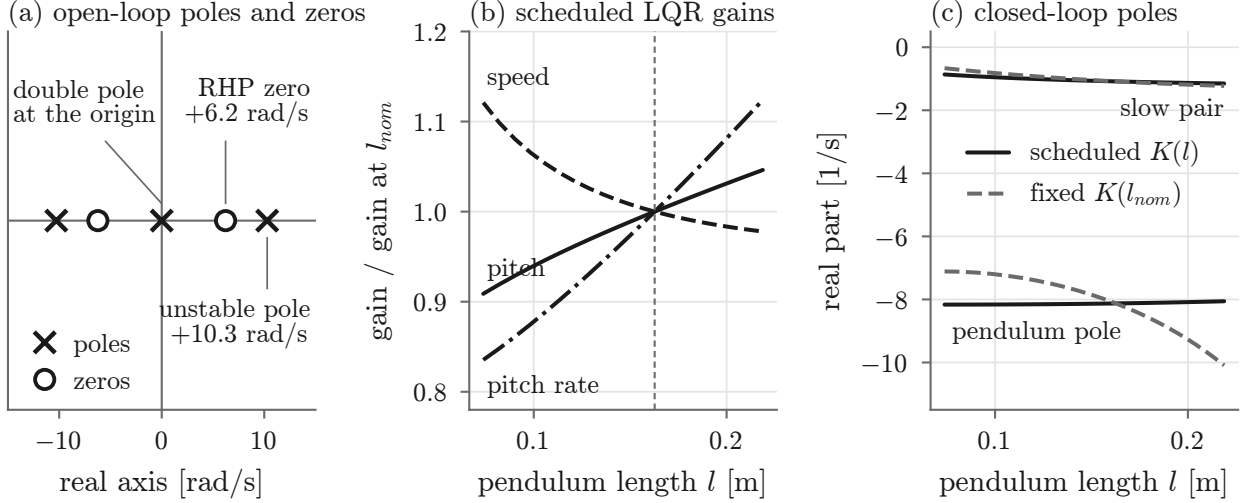


Figure 5: (a) Poles and zeros of the linearised pendulum, from the common torque to the position, in the nominal posture: one unstable pole and one right-half-plane (RHP) zero. (b) LQR gains of the common torque on speed, pitch and pitch rate over the 25 postures of the schedule, relative to their nominal value; the gain on the position does not change. (c) Dominant closed-loop poles with the scheduled gain and with the nominal gain kept fixed.

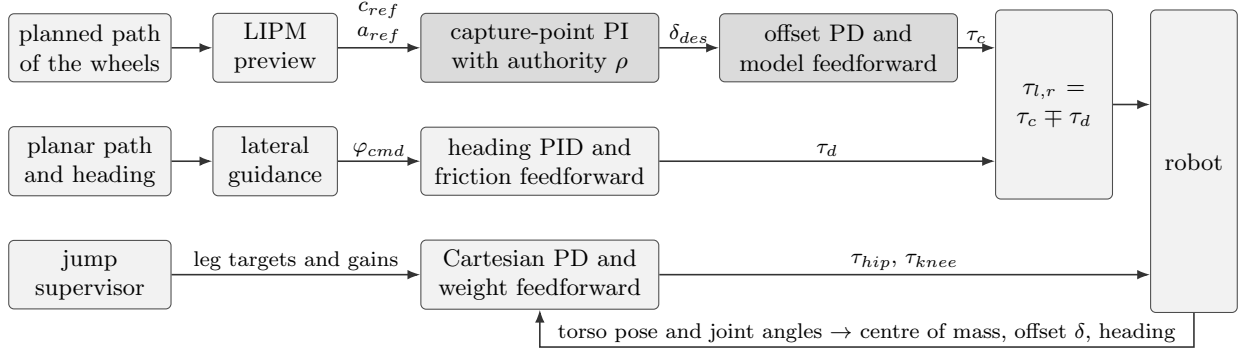


Figure 6: Architecture of the cascaded PID. The two shaded blocks form the sagittal cascade: the outer one decides where the contact should be with respect to the centre of mass, the inner one makes the wheels realise it. The measurements reach every block; a single feedback path is drawn for clarity.

bounds but the natural *input* of the balance: rolling the wheels moves it, and by (4) the offset $\delta = c - p$ between CoM and ZMP sets the acceleration of the CoM. This is the cart-table view of Kajita *et al.* [5], read for a robot whose “table” is a pair of wheels.

The controller is organised accordingly (Fig. 6). An outer loop decides which offset is needed; an inner loop makes the wheels realise it; the heading uses the differential torque; the legs keep the posture. All the quantities it needs—the CoM, the contact line, the heading—are computed with the model library from the pose of the torso and the joint angles. In this controller the pose of the torso is taken from the simulator; the consequences of this choice for the comparison are discussed in Section 6.1.

4.2 The sagittal cascade

Outer loop: the capture point. The linear pendulum has one stable and one unstable mode, and the unstable one is carried by the *capture point* $\xi = c + \dot{c}/\omega$, the point where the contact would have to be for the robot to come to rest [6]. Its dynamics is of first order and is driven directly by the ZMP:

$$\dot{\xi} = \omega (\xi - p). \quad (5)$$

Controlling the capture point therefore controls the only part of the motion that diverges, and it requires nothing but a choice of p [7]. Let e_c be the error of the CoM with respect to its reference and $e_\xi = e_c + \dot{e}_c/\omega$ the corresponding error of the capture point. Asking e_ξ to decay at a rate k_ξ , with an integral term to absorb constant disturbances, and solving (5) for the contact gives the offset that the wheels should produce:

$$\delta_{des} = \delta_{ff} - \frac{\rho}{\omega} \left(\dot{e}_c + k_\xi e_\xi + k_I \int e_\xi dt \right), \quad |\delta_{des}| \leq \frac{a_{max}}{\omega^2}. \quad (6)$$

The term δ_{ff} is a feedforward, discussed below, and $\rho \in (0, 1]$ is the *authority* of the loop, whose role is the subject of Section 4.4. The bound limits the acceleration that the controller may ask for to $a_{max} = 3 \text{ m/s}^2$; for RoTino this is an offset of about 6 cm, that is a lean of more than 20° . The integral is itself bounded, as an anti-windup measure. Both bounds will be visible in the results.

Inner loop: the offset. The wheels have to place the contact so that the measured offset δ follows δ_{des} . A PD on the offset, added to a feedforward torque, gives the common wheel torque:

$$\tau_c = \tau_{ff} + K_\delta (\delta - \delta_{des}) + D_\delta (\dot{\delta} - \dot{\delta}_{ff}). \quad (7)$$

The sign is the one that the non-minimum-phase plant requires: if the CoM is too far ahead of the contact, the wheels are driven forward to catch up with it.

Feedforward from the model. To hold a constant acceleration a without changing its lean, the pendulum (3) needs a definite lean and a definite torque, obtained by setting $\ddot{\theta} = 0$, $\ddot{s} = a$. The solution is proportional to a : about 22 mm of offset and 0.17 Nm per wheel for each m/s^2 . The offset is close to the value h/g of the simple pendulum (4); the small difference is the contribution of the wheel inertia, which the VL-WIP model includes. These two constants give δ_{ff} and τ_{ff} from the reference acceleration, so the feedback only has to correct what the model does not predict.

4.3 Preview of the planned path

What reference should the CoM follow when the wheels are asked to follow a path $p_{ref}(t)$? Because of the right-half-plane zero, it cannot be the path itself: the CoM must already be ahead of the wheels when they start to accelerate. The reference is the solution of the pendulum equation $\ddot{c}_{ref} = \omega^2(c_{ref} - p_{ref})$ that stays bounded, and this solution is not causal: it is the planned path smoothed by a two-sided exponential kernel,

$$c_{ref}(t) = \frac{\omega}{2} \int_{-\infty}^{+\infty} e^{-\omega|u|} p_{ref}(t+u) du. \quad (8)$$

The CoM reference thus looks about $1/\omega \approx 0.14 \text{ s}$ ahead and behind (Fig. 7a); in the code the integral is a weighted sum over a window of the planned path. The reference acceleration $a_{ref} = \omega^2(c_{ref} - p_{ref})$ that enters the feedforward is a smoothed version of the acceleration of the path that starts *before* it (Fig. 7b): the robot begins to lean a few tenths of a second before the wheels are asked to move. This is the ZMP preview of walking pattern generators [5], applied here to a known path. When the robot is driven by live commands the future is unknown, and the feedforward falls back on the current, rate-limited acceleration of the command.

4.4 Why the outer loop has reduced authority

With $\rho = 1$, (6) is the textbook capture-point controller. It rests on an assumption: that the contact can be placed instantly where the outer loop wants it. Here this is not true. The inner loop moves the contact by accelerating a wheel against a pendulum whose own dynamics is at about

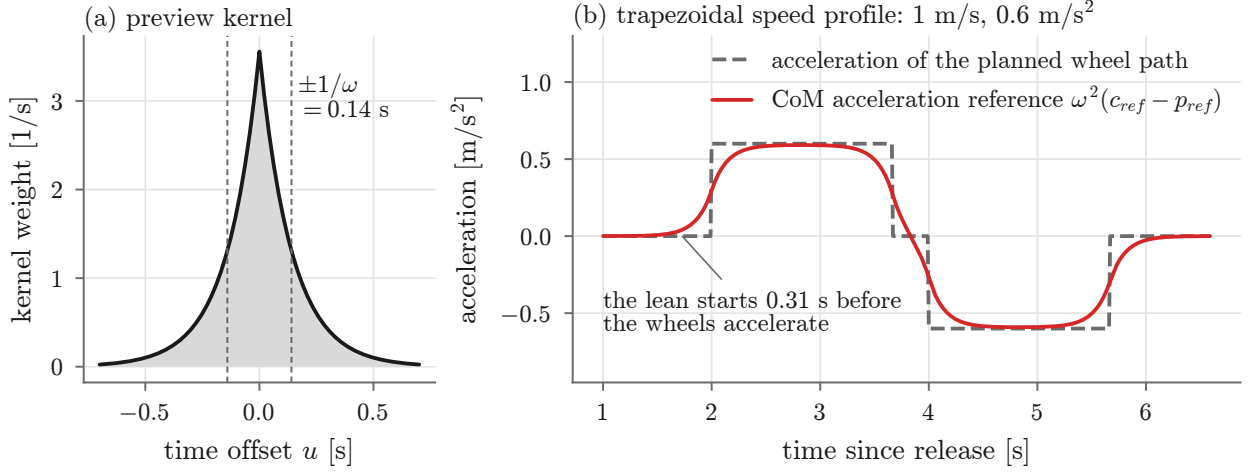


Figure 7: The preview of the planned path. (a) The kernel that turns the planned wheel path into the reference of the centre of mass. (b) For a trapezoidal speed profile, the acceleration of the path is a sequence of steps; the acceleration reference obtained with the preview anticipates each of them.

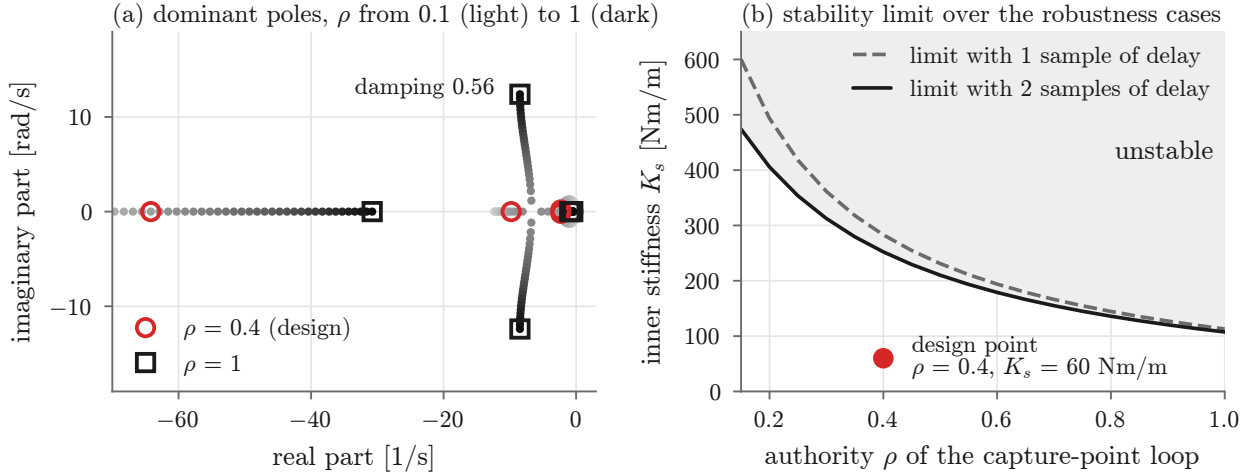


Figure 8: Choice of the authority ρ of the capture-point loop on the sampled linear model. (a) With full authority an oscillatory pair appears; with $\rho = 0.4$ the dominant poles are real. (b) Largest inner stiffness for which all the robustness cases remain stable: the lower the authority, the wider the margin.

10 rad/s, the control runs at 500 Hz, and one or two samples of delay separate a measurement from the torque it produces.

To choose the gains, the cascade (6)–(7) was closed on the linear model (3), sampled at 500 Hz with one and two samples of delay, for ten cases: the nominal robot, an upper body 20 % lighter and heavier, and the legs folded and stretched. Figure 8 summarises what this analysis shows. With full authority the dominant poles include an oscillatory pair, and the inner stiffness K_δ can be less than doubled before some of the ten cases become unstable. Reducing the authority brings those poles back to the real axis and widens the stable range of K_δ . The value $\rho = 0.4$ is where the dominant poles are best damped; the stiffness chosen for the inner loop, 60 Nm per metre of offset, then has a margin of about four. The price is that the outer loop corrects a position error more slowly, and the integral term is there to recover what the reduced authority leaves. The same linear model, driven by the actual control code, is used by the unit tests of the package: tracking of the trapezoid, rejection of the push of the benchmark and removal of an unknown offset of the CoM are all checked offline before the robot is simulated.

4.5 Heading and lateral guidance

The yaw equation in (3) is a double integrator driven by the differential torque, so a PID on the heading error is adequate, with one addition. When the robot turns, its wide wheels scrub on the ground. Without compensation the yaw advances in jerks (stick-slip), and the lateral dynamics of the robot no longer follows the planned turn. A feedforward, computed from the *reference* yaw rate so that it cannot chatter, supplies the Coulomb and viscous friction torque identified in simulation.

To follow a path on the plane, the heading reference is corrected by the lateral error e_y of the axle with respect to the path:

$$\varphi_{cmd} = \varphi_{ref} - \arctan(k_y e_y) \cdot \min(1, |v_{ref}|/v_0). \quad (9)$$

The robot steers towards the path by an angle that grows with the error and saturates, and the correction fades out as the reference speed goes to zero. The last factor is the practical face of the nonholonomic constraint (1): a differential drive cannot move sideways, so a lateral error can only be removed while travelling, and a smooth time-invariant feedback cannot regulate the full pose at standstill [8]. The path itself is planned as in the course, with a geometric path and a separate timing law: the S-curve is a cubic polynomial $y(x)$ that leaves and arrives parallel to the initial heading, travelled with a quintic law in the arc length.

The common and the differential torque are finally mixed as $\tau_{l,r} = \tau_c \mp \tau_d$. When the sum would exceed the torque limit, it is the common torque that keeps priority: a robot that turns late is a nuisance, one that falls is a failure.

4.6 Legs and jump supervisor

In this controller the legs are a positioning device: they hold the wheel under the hip at the commanded leg length. Each leg has a Cartesian PD on the position p of the wheel centre with respect to the hip, with the share of the weight it carries as feedforward, mapped to joint torques by the transpose of the leg Jacobian J :

$$\tau_{leg} = J^T (K_p (p_d - p) - K_d \dot{p} + F_{ff}). \quad (10)$$

Working in Cartesian space was the remedy for the limit cycle mentioned in the introduction. The joint-space gains inherited from the position-servo controller were so high that, at 500 Hz, a single control step could reverse the velocity of the light leg links; the Cartesian gains correspond to a joint stiffness about three times lower, and the oscillation disappears.

The jump is handled by a supervisor with seven states: balance, preload (a squat), thrust (a fast extension), flight, landing, recovery and settling. It changes the leg targets and gains from one phase to the next and scales the authority of the capture-point loop, which is reduced during thrust and landing and removed in flight, when the wheels are only damped. The transitions that depend on contact—take-off and touchdown—combine the contact sensors, filtered with hysteresis and a short debounce, with the clearance of the wheels and the vertical velocity of the CoM. A robot that jumps is a hybrid system, and a single spurious contact reading at the wrong moment would switch the control law in mid-air.

A compensation of the *lateral* ZMP was also implemented: in a turn the legs are made unequal in length so that the torso leans into the curve, as a cyclist does, with the same preview as in the sagittal plane. It is disabled by default, because with the rigid cylindrical wheels of the model the lean moves the contact onto the wheel edge and disturbs the yaw; the results below show that the lateral margin is in any case large.

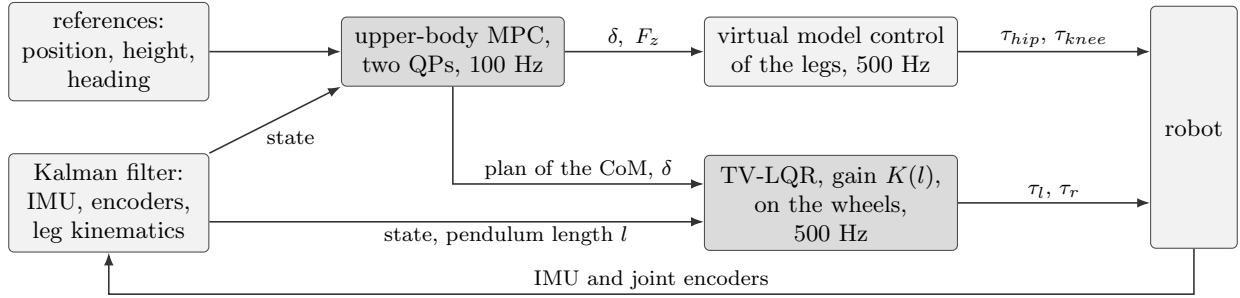


Figure 9: Architecture of the second controller. The MPC plans the motion of the upper body; the LQR, whose gain follows the length of the pendulum, makes the wheels follow that plan; virtual model control realises the planned offset and vertical force with the legs.

5 Second controller: MPC, TV-LQR and virtual model control

5.1 The architecture of the paper and its implementation

Cui *et al.* [1] place their method between two extremes: controllers that treat the robot as a plain wheeled pendulum and ignore the legs and the torso, and whole-body controllers, which are complete but complex. Their proposal assigns each of the two models of Section 3.3 to the controller that suits it. Balance is fast and unstable, but it is well described by a linear model, so it is given to an LQR on the wheels. The motion of the upper body is slower, but it is subject to limits—friction, leg length, available force—so it is given to an MPC, which can handle constraints. The MPC does not command torques: it decides the offset δ of the CoM and the vertical force F_z , and virtual model control turns them into leg torques. A Kalman filter provides the state. Figure 9 shows the implementation in `rotino_mpc`.

Unlike the PID of the previous section, this controller uses only the sensors that a real robot would carry: the IMU and the joint encoders. The true pose published by the simulator is read only to log the estimation error.

5.2 State estimation

The estimator answers the question that the PID avoided: where is the torso, and how fast is it moving, if the only data are those of the robot itself? Two sources are available, with opposite defects. The accelerations of the IMU, rotated to the world frame and integrated, are accurate over short times but drift. The wheels, through the rolling constraint (1) and the kinematics of the legs, give the position and the velocity of the torso without drift, but they are wrong whenever a wheel slips or leaves the ground. As in the paper, a linear Kalman filter merges them. For each axis the state is the position and the velocity of the torso; the prediction integrates the measured acceleration a over the sampling time T , and the correction compares the result with the position and velocity obtained from odometry:

$$\begin{bmatrix} p \\ v \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ v \end{bmatrix}_k + \begin{bmatrix} T^2/2 \\ T \end{bmatrix} a_k, \quad y_k = \begin{bmatrix} p_{axle} + p_{torso/axle} \\ v_{axle} + v_{torso/axle} \end{bmatrix}. \quad (11)$$

At steady state the filter behaves as a complementary filter: odometry prevails at low frequency, the IMU above a few hertz. Because the three axes are decoupled, the implementation runs three independent two-state filters, which is equivalent and cheaper. During the flight phase of a jump the wheels carry no information, and the correction is simply skipped; the paper obtains the same effect by inflating the measurement noise. The lean θ and the length l of the pendulum do not need a filter: they follow from the orientation given by the IMU and from the leg kinematics.

5.3 Balance: a gain-scheduled LQR

The wheels are driven by a state feedback on the model (3). With $\tilde{x}$ the error of the state $x = (s, \theta, \varphi, \dot{s}, \dot{\theta}, \dot{\varphi})$ with respect to its reference and $u = (\tau_l, \tau_r)$, the gain minimises

$$J = \int_0^\infty (\tilde{x}^T Q \tilde{x} + u^T R u) dt, \quad u = -K(l) \tilde{x}. \quad (12)$$

The model depends on the length of the pendulum, hence so does the gain: this is what the paper calls a time-varying LQR. In the implementation the Riccati equation is solved once, at start-up, for 25 leg postures between a deep squat and full extension, and the gain is interpolated on the current value of l . The solver is part of the package: it builds the Hamiltonian matrix of the problem and extracts the solution from its stable invariant subspace.

Three aspects of the design deserve a comment.

The weights. Q penalises the lean far more than the position, so that balance has priority over tracking. R is not chosen in the coordinates of the two motors but in those of the common and of the differential torque, with the differential one weighted fifty times more. The reason came from simulation: the torso, held by the virtual springs of the legs, has a torsional resonance at about 12 Hz, and a stiff yaw loop excites it into a sustained oscillation. The heavy weight keeps the yaw loop slow and well below that frequency. The cost of this choice appears in the results on curved paths.

The effect of scheduling. Figure 5b–c shows what the schedule does on a robot of this size. Over the whole range of leg lengths the gains change by about 15 %, and they keep the dominant closed-loop poles almost exactly where they were designed. With the nominal gain kept fixed the loop remains stable, but its poles move with the posture. For RoTino scheduling is therefore a refinement that makes the response independent of the height, rather than a necessity as it is for the large robot of the paper.

The reference. The LQR is not asked to follow the trajectory requested by the operator. Its reference is built from the output of the MPC,

$$x_{ref} = (c_{plan} - \delta, \arctan(\delta/z_{ref}), \varphi_{cmd}, \dot{c}_{plan}, 0, \dot{\varphi}_{ref}) : \quad (13)$$

the lean is the one that corresponds to the planned offset at the reference height z_{ref} , and the position and the speed are those that the MPC predicts for the CoM in the next instants, the position being shifted back by δ because the wheels must stay behind a CoM that leans forward. The LQR thus tracks a plan that already respects the limits of the robot, and it never receives a reference that it could follow only by violating them. The friction feedforward and the slow integral on the heading described in Section 4.5 are added to the differential torque here as well.

5.4 Upper body: a constrained MPC

The MPC works on the lumped-mass model (4). Over a horizon of 25 steps of 20 ms, half a second in all, it chooses the sequences of the offset δ and of the vertical force F_z that minimise the deviation of the CoM from its reference and the control effort,

$$\min \sum_{i=1}^N \left[(x_i - x_i^{ref})^T S (x_i - x_i^{ref}) + w u_i^2 \right] \quad \text{subject to} \quad |\delta| \leq \delta_{max}, \quad F_{min} \leq F_z \leq F_{max}, \quad (14)$$

and applies the first element of each sequence. It runs at 100 Hz, once every five control steps.

Two small problems instead of one. Over the horizon the factor $(g + \ddot{z})/h$ of (4) is frozen at its current value. The horizontal and the vertical dynamics then become two independent double

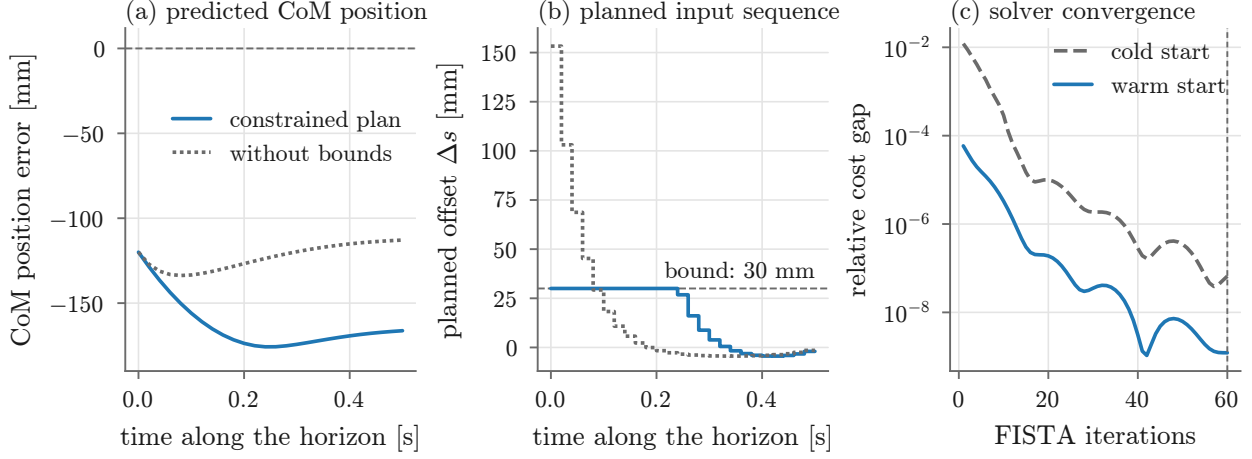


Figure 10: The horizontal MPC when the robot has been pushed backward. (a,b) The plan saturates the offset at its bound for half of the horizon and recovers more slowly than the unconstrained solution would. (c) Convergence of the solver; the controller stops at 60 iterations.

integrators, one driven by δ and one by F_z , and (14) splits into two problems with a single input each. Each double integrator is discretised exactly rather than with the forward-Euler rule of the paper, which costs nothing and removes a source of error at a step of 20 ms.

The constraints, and which one matters. The paper bounds the offset with the friction limit, μh , and with the reach of the leg. For RoTino neither is ever active. With the friction coefficient of the simulation the first bound would allow the CoM to be 36 cm away from the wheels of a robot whose pendulum is 16 cm long, and the second allows 18 cm. A third, much tighter bound was therefore added: $\delta_{max} = 3$ cm, which corresponds to a lean of about 10° and to an acceleration of a little less than 2 m/s^2 . This is the constraint that shapes the behaviour of the controller: whatever happens, the plan never asks the robot to lean more than that. The vertical force is bounded between a fraction and a multiple of the weight of the upper body, so that the wheels neither lift off nor are pressed beyond what the legs can deliver; the upper bound is raised during the thrust of a jump.

Solving the problem. Eliminating the states (*condensing* [10]) leaves a quadratic program in the 25 inputs alone, whose only constraints are bounds on the variables. A problem of this kind does not need a general solver: the projection onto the bounds is a simple clipping, so a projected gradient method is enough. The package implements its accelerated version (FISTA [11]), started from the previous solution shifted by one step. Figure 10 shows a plan in which the bound is active and the convergence of the solver: with the warm start, a few tens of iterations bring the cost to within a negligible distance of the optimum, and a complete update takes about a millisecond in Python, a tenth of the period available.

5.5 Legs: virtual model control

The MPC outputs a displacement and a force, and the legs must realise them. Virtual model control [9] imagines a spring, a damper and a constant force acting between the hip and the wheel, computes the force that these components would produce and maps it to the joints with the transpose of the leg Jacobian. The law has the same form as (10); the difference lies in what the components are asked to do. Horizontally there is a spring: it holds the wheel at the distance δ behind the CoM, which is how the planned offset is realised. Vertically there is *no* spring, only a light damper, and each leg pushes with half of the planned force F_z . The height is therefore not held by a stiffness: it is regulated by the MPC through the force.

Table 4: The architecture of Cui *et al.* and its implementation on RoTino.

	Paper	This implementation
Model parameters	table of the 80 kg prototype; rod inertia $m_b l^2/3$	computed from the URDF; real inertia of the upper body for each posture
LQR gain	function of the pendulum length	25 gains computed at start-up and interpolated; own Riccati solver
LQR weights	weight matrices Q, R	common and differential torque weighted separately, to avoid a leg resonance
MPC model	one model, forward-Euler discretisation	two decoupled double integrators, exact discretisation
MPC constraints	friction, leg reach, vertical force	the same, plus a 3 cm bound on the offset, the only one active on this robot
QP solver	not specified	projected accelerated gradient with warm start
Kalman filter	one planar filter, larger noise in flight	three single-axis filters, no correction in flight
Heading	LQR only	LQR with friction feedforward and a slow integral
Rates	100, 500 and 1000 Hz	MPC at 100 Hz, everything else at 500 Hz

Seen from the ground, each leg behaves as a force-controlled suspension rather than as a rigid strut. This is the Cartesian impedance idea used in the course to make a legged robot compliant at its contacts, with the vertical stiffness set to zero and the weight supplied as a feedforward. Its consequences on uneven ground are among the clearest results of the next section. The jump uses the same blocks: the height reference squats and then accelerates upward so that the CoM leaves the ground at a chosen speed, the force bound is raised for the thrust, and when contact is lost a virtual spring retracts the wheels under the body and holds them there until touchdown.

5.6 Summary of the differences from the paper

Table 4 collects the points in which the implementation departs from [1]. None of them changes the architecture; they adapt it to a robot almost twenty times lighter, to torque control in simulation and to the choice of writing every numerical block in the workspace.

6 Benchmark and results

6.1 Method

The benchmark runs a catalogue of twelve scenarios with both controllers, one after the other, each from a clean simulation: standing balance, an impulsive push, a constant force, a trapezoidal speed profile, a back-and-forth motion, a change of height, a slow and a fast S-curve, the three uneven platforms and a jump. World, robot, torque interfaces and scenario arguments are identical; a logger records every signal at 500 Hz, and the comparison is rebuilt from the logs. The four tests with which the paper validates its controller—change of height, recovery from an impact, speed tracking and jump—are all in the catalogue, the speed profile with the same acceleration; the curved paths, the constant force and the uneven ground were added to stress what those tests leave out. The results below come from one complete execution of the suite.

Reconstructing the ZMP. Since the simulator does not report contact forces, the centre of pressure cannot be measured. The ZMP is therefore computed from its definition, summing the

contribution of every link i of mass m_i and position (x_i, y_i, z_i) , on flat ground:

$$x_{zmp} = \frac{\sum_i m_i [(\ddot{z}_i + g) x_i - \ddot{x}_i z_i] - \sum_i \dot{L}_{y,i}}{\sum_i m_i (\ddot{z}_i + g)}, \quad (15)$$

and likewise for y_{zmp} , where L_i is the angular momentum of the link about its own centre. Positions come from the true pose of the torso and from the joint angles through the model library; accelerations are obtained by fitting a parabola to a short window of samples, which avoids differentiating noise twice. The same code runs offline on the logs and online in the dashboard. Two quantities are derived from it: the lateral position of the ZMP as a fraction of the half-distance between the wheels, where 100 % means that the inner wheel is about to lift, and the load on each wheel. In the direction of travel the ZMP must lie on the contact line, which provides a check: on flat ground the reconstructed point stays within a millimetre of it.

How to read the comparison. Three cautions apply. First, the PID reads the pose of the torso from the simulator, whereas the other controller estimates it; a perfect measurement favours the PID in smoothness. Second, for the same reason the position that each controller logs is not the same signal. All the positions shown below are therefore recomputed for both from the true pose, and so are the heights and the wheel clearances. This matters: on straight paths the tracking error computed in this way is similar for the two laws, whereas the internal logs would suggest a clear advantage of the MPC, simply because its error is measured against its own odometry, which can overestimate the distance travelled by up to about one per cent. Third, each scenario was run once, and small differences fall within the variability from run to run. For these reasons only the differences that are large, and that can be explained from the design of the controllers, are discussed.

6.2 Disturbances: push and constant force

The push is a short impulse that sends the torso backward at about 0.6 m/s (Fig. 11a). The two controllers react in opposite ways, and the middle panel shows why. The PID asks at once for the largest offset that it allows, 6 cm: the robot leans by about 20°, brakes hard, drifts by about 20 cm, and its pitch is back at rest in about two seconds with a single small overshoot. The MPC is bound to 3 cm. It holds the offset on the bound for almost a second, so the lean never exceeds about 11°, but the braking is gentler: the robot drifts by about 35 cm and needs a few oscillations to settle. Neither behaviour is right or wrong. They are the direct expression of the two limits written in the controllers, a_{max} in (6) and δ_{max} in (14), and they show what a constraint in the plan achieves: the MPC keeps the posture within a prescribed envelope and accepts a larger displacement in exchange. The linear model of Section 4.4, driven by the control code, predicts a peak of about 19° for the PID: the simple model is a good guide to the simulated robot.

The constant force (Fig. 11b) asks a different question: what remains once the transient is over? Both robots end up leaning into the force by the same angle, as statics demands. What differs is where they stand. The PID stops about 18 cm from the starting point, the MPC about 10 cm. One might expect the integral term of the PID to cancel the error. It does not, because the lean that balances this force requires an integral larger than its anti-windup bound: the integral saturates, and the rest of the offset must come from the proportional term, that is from a position error. Inserting the gains of the code in (6) gives 18 cm, exactly what is measured. The MPC has no integral action at all; its steady error is the point where the offset that its cost function is willing to command for that position error equals the one needed to hold the force. A disturbance observer, such as the momentum-based estimator presented in the course, would remove the error in both controllers and is the most natural extension of this work.

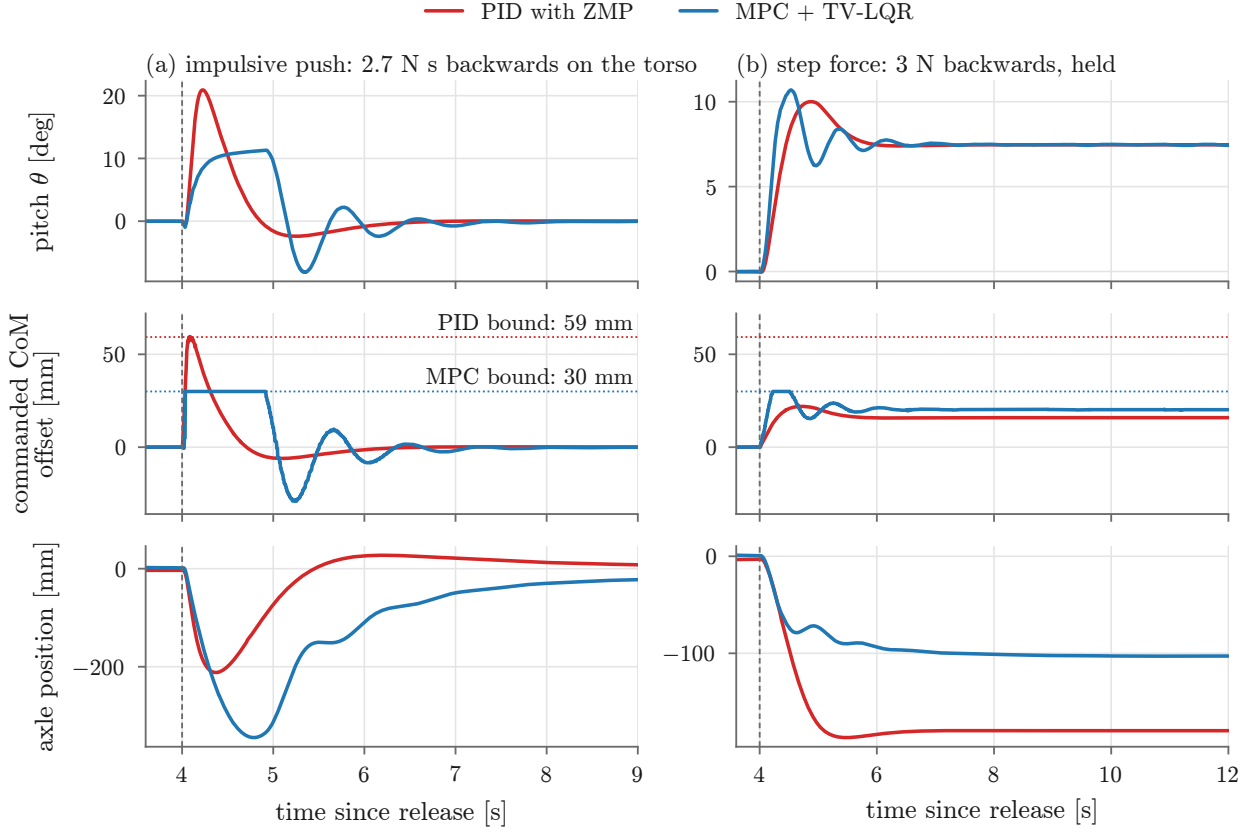


Figure 11: Response to disturbances applied to the torso four seconds after the release. Top: pitch of the equivalent pendulum. Middle: offset between centre of mass and contact commanded by each controller, with the bound that each imposes. Bottom: true position of the axle; the robot is asked to stay where it is.

6.3 Tracking a planned motion

In the trapezoid the robot must accelerate, cruise and brake (Fig. 12). The effect of the preview is visible before the motion starts: when the reference begins to move, the PID is already leaning by a few degrees (panel c). Its pitch then follows the shape that the plan requires—forward during the acceleration, level while cruising, backward while braking—with hardly any overshoot, and it settles about a second after the stop. The MPC does not know the acceleration step in advance in the same way: its horizon sees the reference half a second ahead, but the cost only penalises the deviation of the CoM from it. Its pitch reaches the same plateau through an overshoot and a few oscillations, and it needs several seconds to become quiet after braking. The oscillation comes from the LQR following a reference that the MPC updates continuously, and it is the less satisfactory aspect of this controller on RoTino. The position errors of the two laws are of the same size, a few centimetres at most during the transients, although with different shapes. The back-and-forth scenario gives the same picture.

6.4 Curved paths

On the S-curves the sagittal behaviour is that of the trapezoid, and the interest shifts to the plane (Fig. 13). The PID stays within about a centimetre and a half of the path; the MPC deviates by up to 6 cm and ends about 6 cm to its side. The cause is not the estimator. Replaying the odometry on the logged encoder and gyroscope signals reproduces the true final position to within a few millimetres sideways, so the controller knew where it was. The cause is the heading loop. To stay clear of the leg resonance, the LQR was given a yaw stiffness that is half that of the PID, and with the friction of the wheels on the ground a soft loop leaves a larger heading error; since the guidance (9) fades out as the robot stops, what has not been recovered by then remains. This is

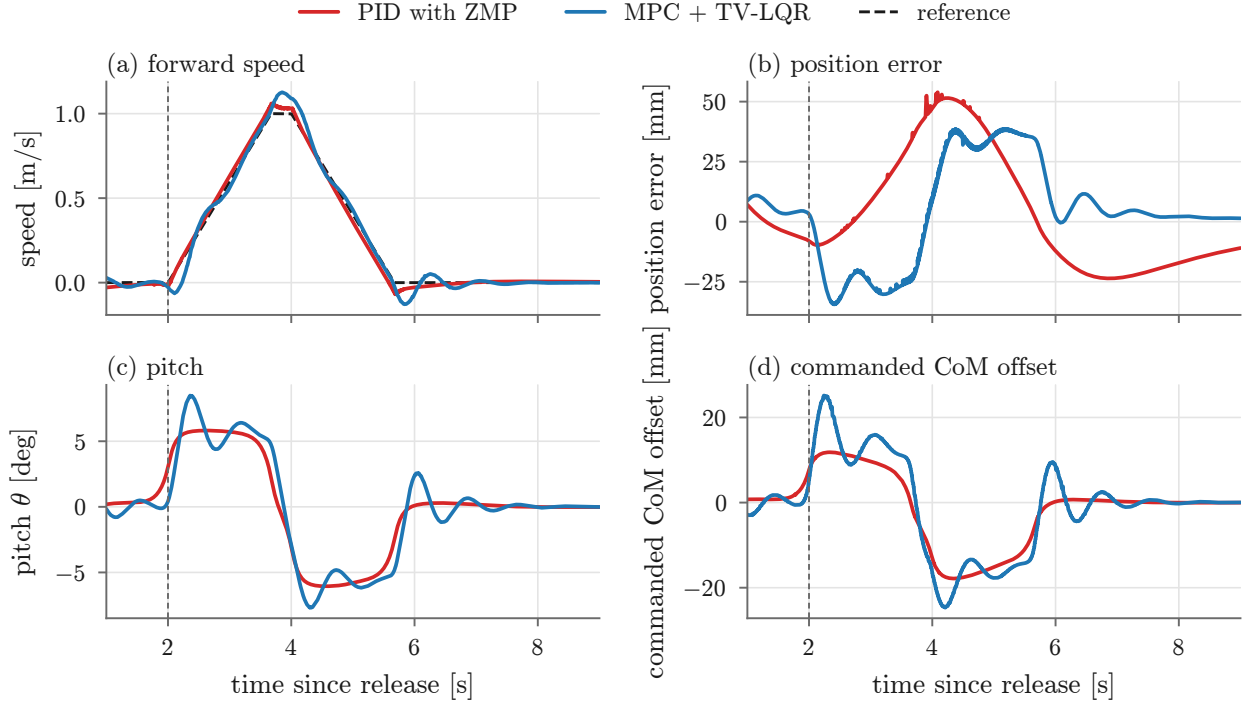


Figure 12: Two metres forward with a trapezoidal speed profile (1 m/s , 0.6 m/s^2). The motion starts at the dashed line; the position error is computed from the true pose for both controllers.

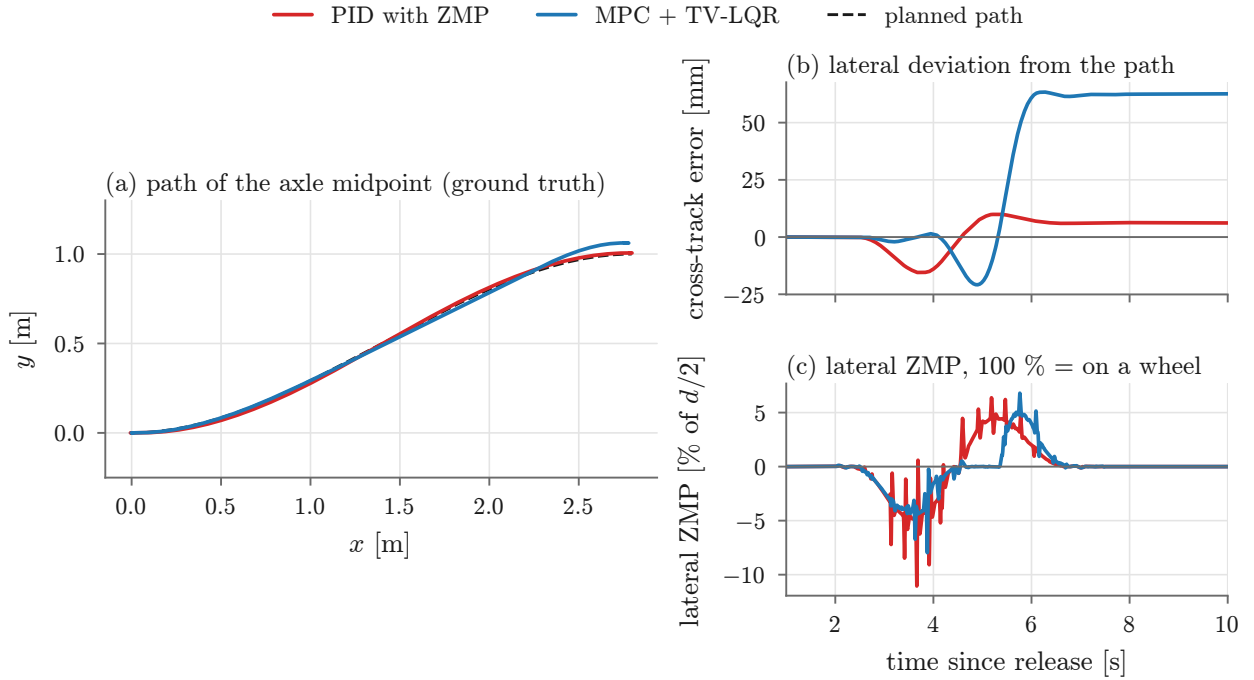


Figure 13: The fast S-curve: three metres of path with a lateral displacement of one metre in five seconds.

the price of the weighting choice of Section 5, and it suggests that the resonance would be better treated at its source, by damping the yaw of the torso with the legs, than by slowing the heading loop down.

The lateral ZMP gives the same message for both controllers (panel c). Even in the fast curve it moves by about a tenth of the half-distance between the wheels: at these speeds the robot is far from tipping, and both wheels keep most of their load. This is why the lateral compensation of the PID can remain switched off.

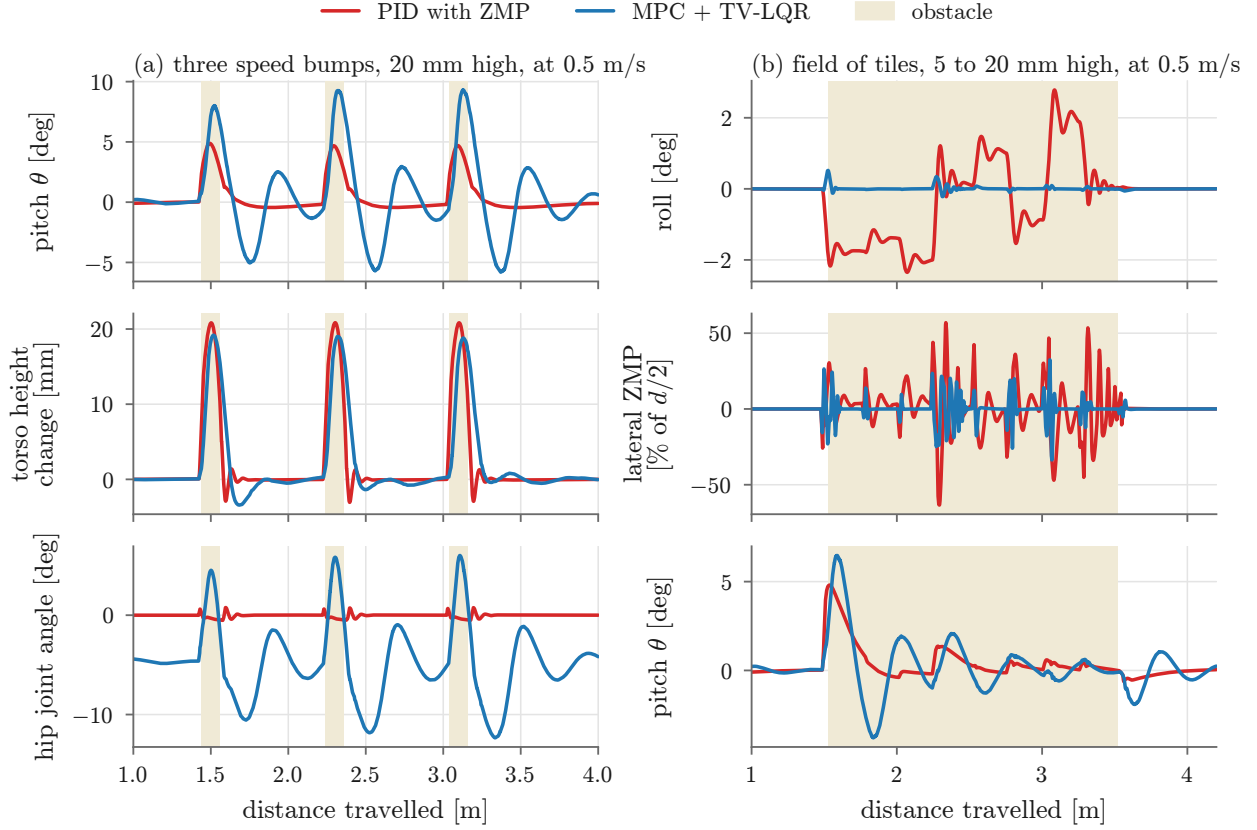


Figure 14: Crossing the speed bumps and the field of tiles at 0.5 m/s, plotted against the distance travelled. The change of height of the torso is computed from its true pose.

6.5 Uneven ground

Neither controller knows anything about the ground: both assume that it is flat, and the platforms test how the two structures absorb what they do not model (Fig. 14). On the ramp both simply climb and descend with a pitch deviation of a degree or two; the more interesting cases are the other two.

Speed bumps. A bump is a brief longitudinal disturbance on both wheels at once. The PID meets it with rigid legs: the hip angle hardly moves, the pitch deviates by about 5° and returns without oscillating. In the MPC the disturbance propagates through the whole chain: the pitch error changes the planned offset, the legs swing by almost 20° to realise it, and the pitch reaches about 9° and oscillates between one bump and the next. In both cases the torso rises by the height of the bump. The logs of the benchmark suggested otherwise, because the MPC estimates its height assuming flat ground and so does not see the bump at all; the comparison in the figure uses the true pose for both.

Tiles. Here the two wheels meet steps of different height, and the roles reverse. With the PID the torso rolls by almost 3° : its legs are stiff position loops, so the difference of height between the two wheels is transmitted to the body, the lateral ZMP is thrown towards one wheel by more than half of the available margin, and the load on the other wheel drops to a third of its static value. With the MPC the roll stays near half a degree, the excursion of the ZMP is roughly halved and the wheels remain better loaded. No part of that controller controls the roll explicitly. The result is a property of virtual model control: each leg pushes with half of the planned force whatever its length, so it lengthens or shortens to follow the step under its wheel. A choice made for another reason—leaving the height to the MPC—turns out to give the robot a suspension.

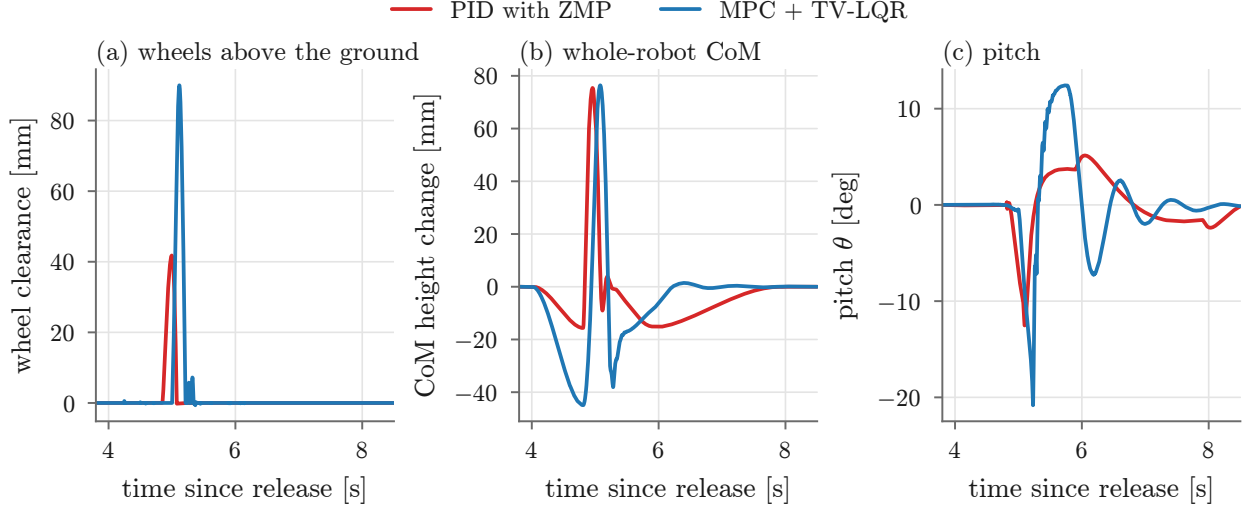


Figure 15: The jump, with wheel clearance and height of the centre of mass computed from the true pose.

Table 5: Summary of the results (one run per scenario, rounded values).

Scenario	Quantity	PID with ZMP	MPC + TV-LQR
Standing balance	wheel torque needed to stand still	very low	three to four times higher
Push	peak pitch	21°	11°
	largest drift	0.2 m	0.35 m
Constant force	steady displacement	0.18 m	0.10 m
Trapezoid	peak pitch	6°	8.5°
	time to settle after the stop	about 1 s	more than 4 s
Height change	pitch deviation	negligible for both	
Fast S-curve	largest deviation from the path	1.5 cm	6 cm
	lateral ZMP, share of the margin used	about one tenth for both	
Speed bumps	peak pitch	5°	9°
Ramp	peak pitch	below 2° for both	
Tiles	peak roll	3°	0.5°
	lateral ZMP, share of the margin used	about 60 %	about 35 %
Jump	wheel clearance	4 cm	9 cm
	time to settle after landing	about 4 s	less than 3 s

6.6 Jump

Both controllers make the robot jump and recover (Fig. 15). The CoM rises by about 7 cm in both cases and the flight lasts between two and three tenths of a second, but the two jumps look different. The PID replays a fixed sequence of leg postures and holds the take-off posture in the air: the wheels clear the ground by about 4 cm. The MPC squats deeper, pushes with a planned force and retracts the legs in flight, which doubles the clearance of the wheels for the same height of the CoM. Landing is the critical moment. The PID lands softly, because its supervisor lowers the authority of the balance loop during landing and recovery, and then takes about four seconds to settle. The MPC reacts at once and with all the available torque—the wheels reach their limit at touchdown—and after a sharp transient of the pitch it is steady in less than three seconds.

6.7 Overall picture

Table 5 condenses the scenarios into the figures that characterise them, and the tally of the benchmark over all its metrics is balanced: neither law dominates. What the results show is rather a pattern, which follows from the structure of the two controllers.

The PID is at its best when the motion is known in advance. The preview lets it lean at the right moment, its torques are small and smooth, and its pitch follows the plan without oscillating.

Its limits are those of a controller without constraints and with rigid legs: when it is surprised it reacts with all the lean it is allowed, and uneven ground reaches the torso directly. It also relies, in this project, on a perfect measurement of the pose.

The MPC architecture is at its best where limits and contacts matter. The bound on the offset keeps the posture within a known envelope under any disturbance, the force-controlled legs filter the ground, and all of this is obtained using only the sensors of the robot. It pays with a more oscillatory pitch, with a soft heading loop and with more parameters to tune; the computation, often seen as the cost of predictive control, is not an issue at this scale.

A third lesson concerns the comparison itself. Several of the conclusions above differ from those that the raw metrics of the benchmark suggested, because the two controllers measure position and height in different ways. Comparing controllers fairly requires comparing the same physical quantities, and a simulator, which offers the true state, makes this possible—provided that one remembers to use it.

7 Relation to the topics of the course

The project touched most of the chapters of the course, and it is useful to say where.

- *Configuration space and underactuation.* The count of the degrees of freedom of the floating base, the topology of the configuration space and the reading of underactuation as a property of the model (Section 3) apply the introductory part of the course to a robot that changes model three times, from the full floating base to a two-state pendulum.
- *Wheeled robots.* Rolling without slipping gives the Pfaffian constraints (1), the odometry used by the Kalman filter and the reason why the lateral guidance must fade at standstill. The reference paths are planned with a geometric path and a separate timing law. What is new with respect to the course is that the kinematic model is only the outer shell of a dynamic problem.
- *Aerial robots.* RoTino shares with a quadrotor the fact that it translates by tilting: the hierarchical structure, in which an outer loop turns a position error into a desired tilt for a faster inner loop, is the one of the cascaded PID, with the ZMP offset in the role of the thrust direction.
- *Legged robots.* The floating base, the unilateral contact, the friction bound, the ZMP and its relation with the support polygon, and the lumped-mass simplification of the centroidal dynamics are the vocabulary of Sections 4–6. The course formulates the control of a quadruped as a quadratic program with constraints; here a much smaller program is solved over a horizon, and the comparison with a controller that has no constraints shows what they are for. Virtual model control is the Cartesian impedance of the course applied leg by leg, and the jump adds the hybrid nature of contact.

What the project required beyond the course was mainly the capture point and the preview of the ZMP, the gain-scheduled LQR, the condensed formulation and first-order solution of the MPC, and the reconstruction of the ZMP from multibody dynamics.

8 Conclusions

A wheeled biped was modelled, simulated with torque control and balanced with two controllers built on the same physical idea—the horizontal offset between the centre of mass and the contact sets the acceleration—and on opposite philosophies. The cascaded PID treats that offset as the output of a capture-point loop and anticipates planned motions with a preview. The architecture of Cui *et al.* plans it with a constrained MPC and delegates balance to a gain-scheduled LQR and

compliance to the virtual model of the legs. On twelve scenarios neither is uniformly better. The cascade is smoother and more accurate on planned motions; the predictive architecture keeps the posture bounded under disturbances and behaves better on uneven ground. In almost every case the difference could be traced to a specific line of the design: a bound, a weight, a stiffness.

The limits of the work indicate what should come next. The PID should be fed by the same estimator as the MPC, so that the comparison is made on equal measurements, and each scenario should be repeated to quantify the variability. A disturbance observer would remove the steady error under constant forces. The leg resonance that forced a soft heading loop should be damped at its source. Finally, the wheels should be given a crowned profile, so that the lateral ZMP can be controlled by leaning into curves and used as a constraint when turning at higher speed.

References

- [1] Z. Cui, Y. Xin, S. Liu, X. Rong, Y. Li, “Modeling and control of a wheeled biped robot,” *Micromachines*, vol. 13, no. 5, 747, 2022.
- [2] V. Klemm *et al.*, “Ascento: a two-wheeled jumping robot,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2019, pp. 7515–7521.
- [3] M. W. Spong, “Partial feedback linearization of underactuated mechanical systems,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 1994, pp. 314–321.
- [4] M. Vukobratović, B. Borovac, “Zero-moment point — thirty five years of its life,” *Int. Journal of Humanoid Robotics*, vol. 1, no. 1, pp. 157–173, 2004.
- [5] S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Harada, K. Yokoi, H. Hirukawa, “Biped walking pattern generation by using preview control of zero-moment point,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2003, pp. 1620–1626.
- [6] J. Pratt, J. Carff, S. Drakunov, A. Goswami, “Capture point: a step toward humanoid push recovery,” in *Proc. IEEE-RAS Int. Conf. on Humanoid Robots*, 2006, pp. 200–207.
- [7] J. Engelsberger, C. Ott, M. A. Roa, A. Albu-Schäffer, G. Hirzinger, “Bipedal walking control based on capture point dynamics,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2011, pp. 4420–4427.
- [8] R. W. Brockett, “Asymptotic stability and feedback stabilization,” in *Differential Geometric Control Theory*, Birkhäuser, 1983, pp. 181–191.
- [9] J. Pratt, C.-M. Chew, A. Torres, P. Dilworth, G. Pratt, “Virtual model control: an intuitive approach for bipedal locomotion,” *Int. Journal of Robotics Research*, vol. 20, no. 2, pp. 129–143, 2001.
- [10] J. L. Jerez, E. C. Kerrigan, G. A. Constantinides, “A condensed and sparse QP formulation for predictive control,” in *Proc. IEEE Conf. on Decision and Control and European Control Conf.*, 2011, pp. 5217–5222.
- [11] A. Beck, M. Teboulle, “A fast iterative shrinkage-thresholding algorithm for linear inverse problems,” *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009.
- [12] F. Ruggiero, *Field and Service Robotics*, lecture notes, University of Naples Federico II, a.y. 2025/2026.